



Вычислительная геометрия и алгоритмы компьютерной графики

Лекция №15: Моделирование теней

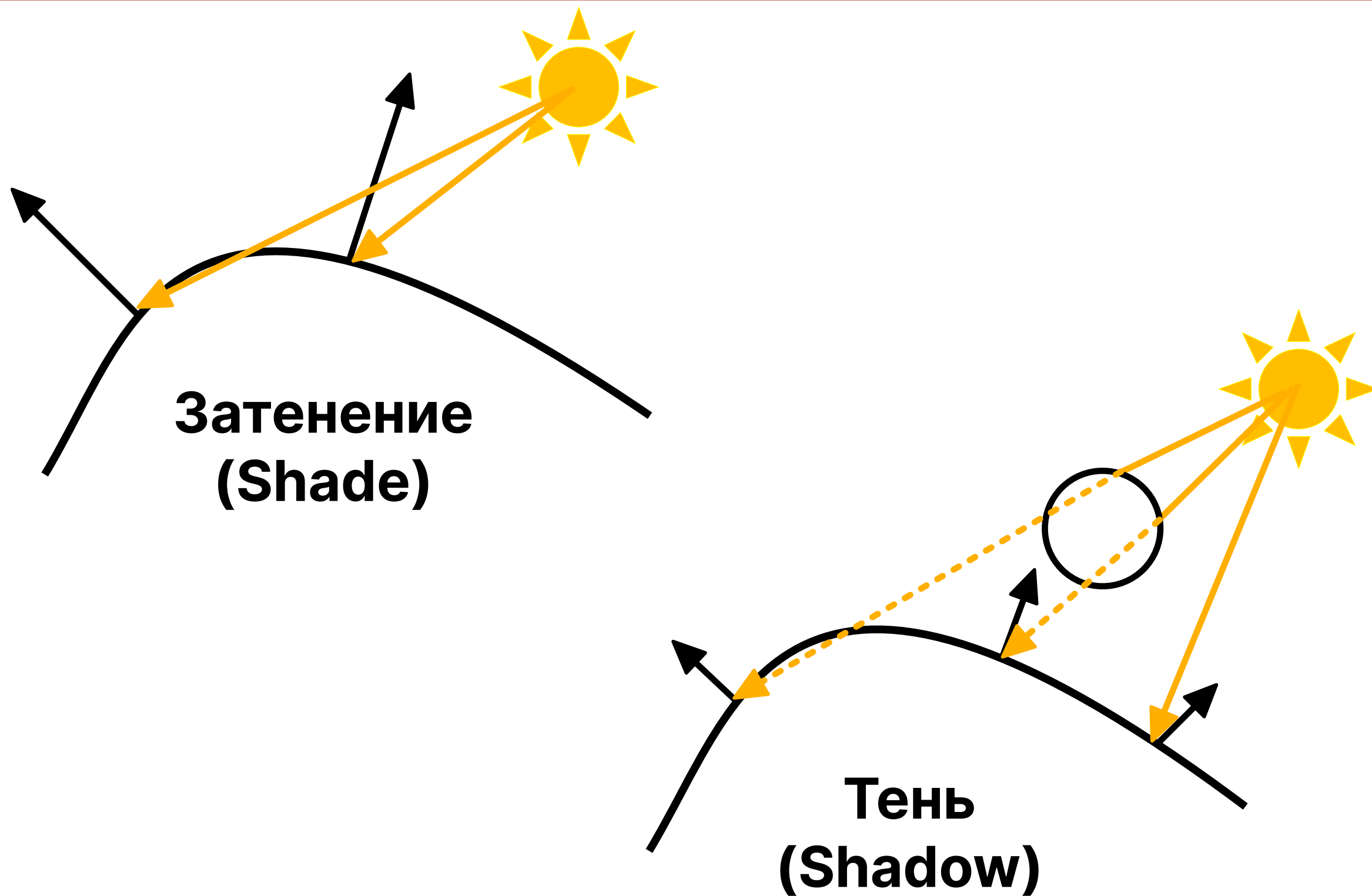
к.ф.-м.н.

Рябинин Константин Валентинович,
kostya.ryabinin@gmail.com

ст. преподаватель

Лабутин Иван Александрович,
linuxrf@gmail.com

Пермь – 2026



Спрайтовая тень:

- **Принцип – расположение под объектом спрайта с текстурой круга или овала (реже – чего-то более сложного)**
- **Контекст использования – мультяшные стилизации**

Пререндеренная карта теней:

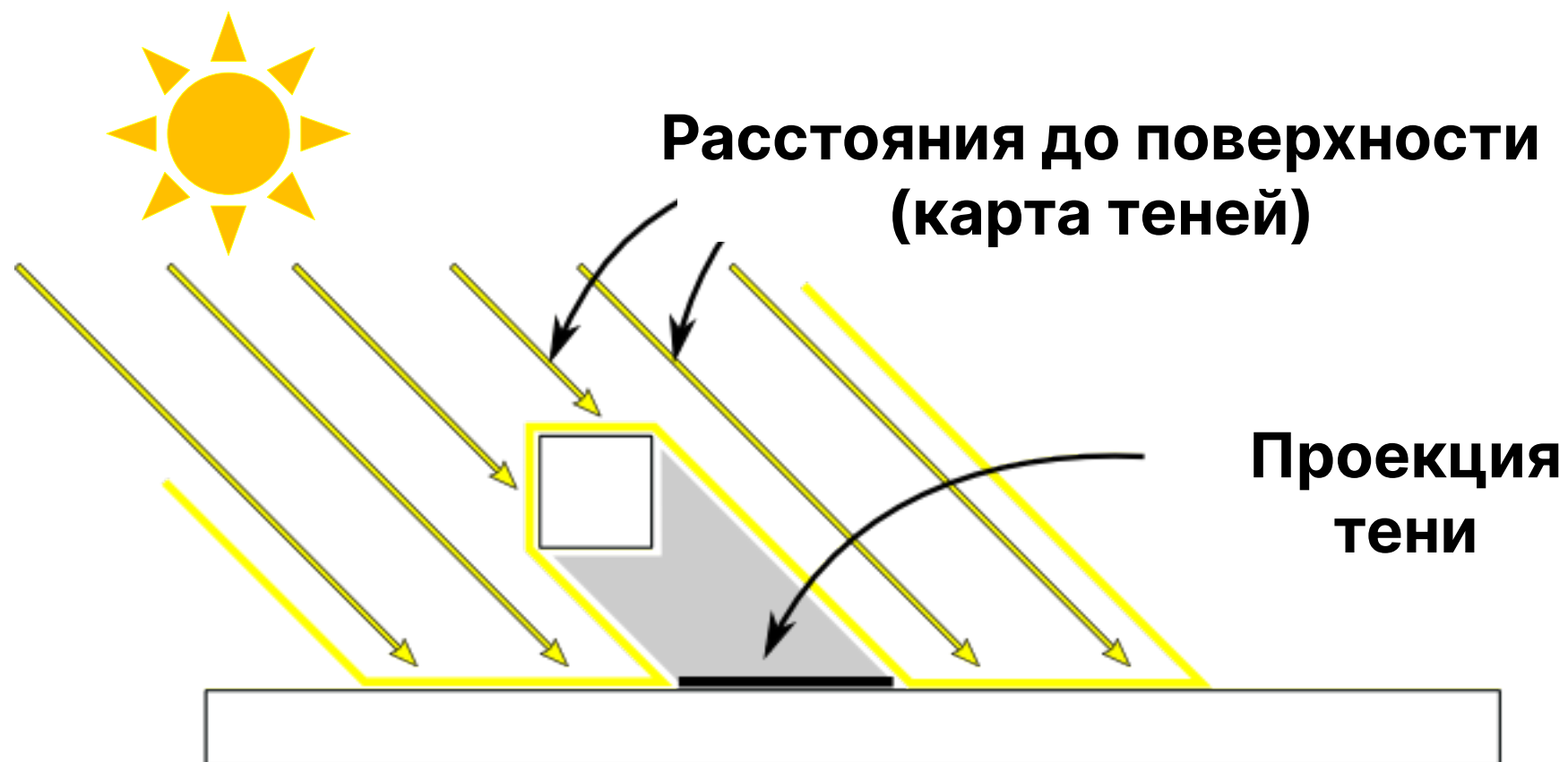
- **Принцип – на поверхности объектов накладывается текстура, содержащая тёмные пятна в нужных местах**
- **Контекст использования – неподвижные объекты и источники света**



Есть много способов создания динамических теней:

- **Трассировка лучей**
- **Теневые объёмы / Трафаретные тени**
<https://developer.nvidia.com/gpugems/gpugems/part-ii-lighting-and-shadows/chapter-9-efficient-shadow-volume-rendering>
- **Фотонные карты в пространстве изображения**
<https://www.cs.jhu.edu/~misha/ReadingSeminar/Papers/McGuire09.pdf>
- **Карты теней / Проекционные тени**

Карта теней – текстура, содержащая расстояния от источника света до ближайшей к нему поверхности



Идея: освещено то, что "видит" источник света

Алгоритм для **направленных** источников:

1. Поместить камеру в позицию источника и ориентировать в его направлении
2. Выполнить визуализацию в текстуру тени, сохраняя не цвета объектов, а расстояния до них (для этого обычно используется специальный тип текстуры)
3. Вернуть камеру в позицию наблюдателя
4. Выполнить визуализацию на экран
5. Для каждого фрагмента:
 - a) Найти расстояние от этой точки до источника света
 - b) Перенести точку поверхности, соответствующую этому фрагменту, в систему координат источника света
 - c) Спроецировать эту точку в текстуру тени
 - d) Сравнить найденное расстояние со значением в текстуре
 - e) Если расстояния равны, фрагмент освещён, иначе – нет

Идея: фактически рендеринг будет идти в z-буфер

// Создание текстуры

```
glGenTextures(1, &shadowTex);
```

```
glBindTexture(GL_TEXTURE_2D, shadowTex);
```

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
```

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
```

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
```

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
```

```
glTexImage2D(GL_TEXTURE_2D, 0, GL_DEPTH_COMPONENT16, 2048, 2048,  
             0, GL_DEPTH_COMPONENT, GL_FLOAT, nullptr);
```

// Создание фреймбуфера

```
glGetIntegerv(GL_FRAMEBUFFER_BINDING, &currentFramebuffer);
```

```
glGenFramebuffers(1, &shadowBuffer);
```

```
glBindFramebuffer(GL_FRAMEBUFFER, shadowBuffer);
```

```
glFramebufferTexture(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, shadowTex, 0);
```

```
glDrawBuffer(GL_NONE);
```

```
glBindFramebuffer(GL_FRAMEBUFFER, currentFramebuffer);
```

Используемый тип проекции зависит от природы источника:

- 1. Лучи света от бесконечно удалённых точечных источников (например, солнце) можно считать параллельными $\Rightarrow$ нужна параллельная проекция**
- 2. Лучи света от прожекторов конечной удалённости расходятся конусом $\Rightarrow$ нужна перспективная проекция**

В самом простом случае:

`lightVP = ortho * lightLookAt`

```
// Рендеринг в текстуру тени
glBindFramebuffer(GL_FRAMEBUFFER, shadowBuffer);
glViewport(0, 0, 2048, 2048);
renderScene(lightVP, shadowPass);
// Рендеринг на экран
glBindFramebuffer(GL_FRAMEBUFFER, currentFrameBuffer);
glViewport(0, 0, screenW, screenH);
renderScene(cameraVP, !shadowPass);
```

Вершинный шейдер должен производить минимум работы, например:

```
#version 330
uniform mat4 u_mvp;
in vec3 a_position;
void main()
{
    gl_Position = u_mvp * vec4(a_position, 1.0);
}
```

Фрагментный шейдер сохраняет расстояния:

```
#version 330
layout (location = 0) out float fragDepth;
void main()
{
    fragDepth = gl_FragCoord.z;
}
```

Задача: спроецировать каждую точку выводимой поверхности в текстуру тени

Для этого нужно:

- 1. Вычислить координаты точки в пространстве источника света, используя вместо cameraVP матрицу lightVP**
Полученные координаты (NDC) будут в диапазоне [-1; 1]
- 2. Линейно отобразить координаты на отрезок [0; 1], чтобы использовать их в качестве текстурных**
Линейное отображение можно осуществить домножением слева на матрицу сдвига-масштаба:
shadowPos = *lightBias* * lightVP * objM * a_pos

$$\mathit{lightBias} = \begin{pmatrix} 0.5 & 0 & 0 & 0.5 \\ 0 & 0.5 & 0 & 0.5 \\ 0 & 0 & 0.5 & 0.5 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

По сути во фрагментном шейдере нужно сделать вычисление вида

```
float lightness = 1.0;  
if (texture(shadowMap, shadowPos.xy).z < shadowPos.z)  
    lightness = 0.1; // Затенение с учётом окружающей подсветки
```

Однако это даст резкую границу тени (алиасинг)

Для более качественного результата можно использовать **PCF** (Percentage Closer Filtering)

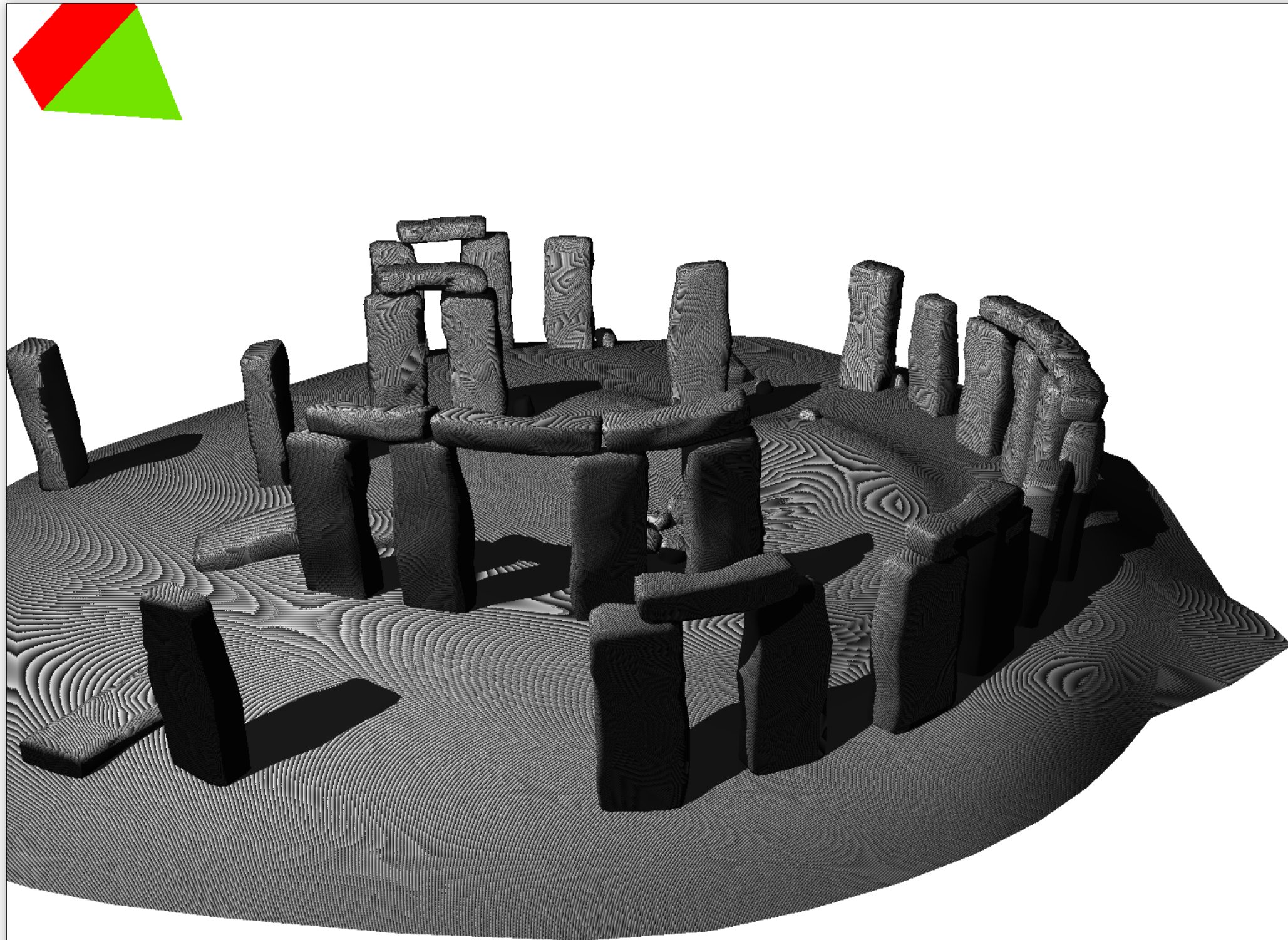
Идея:

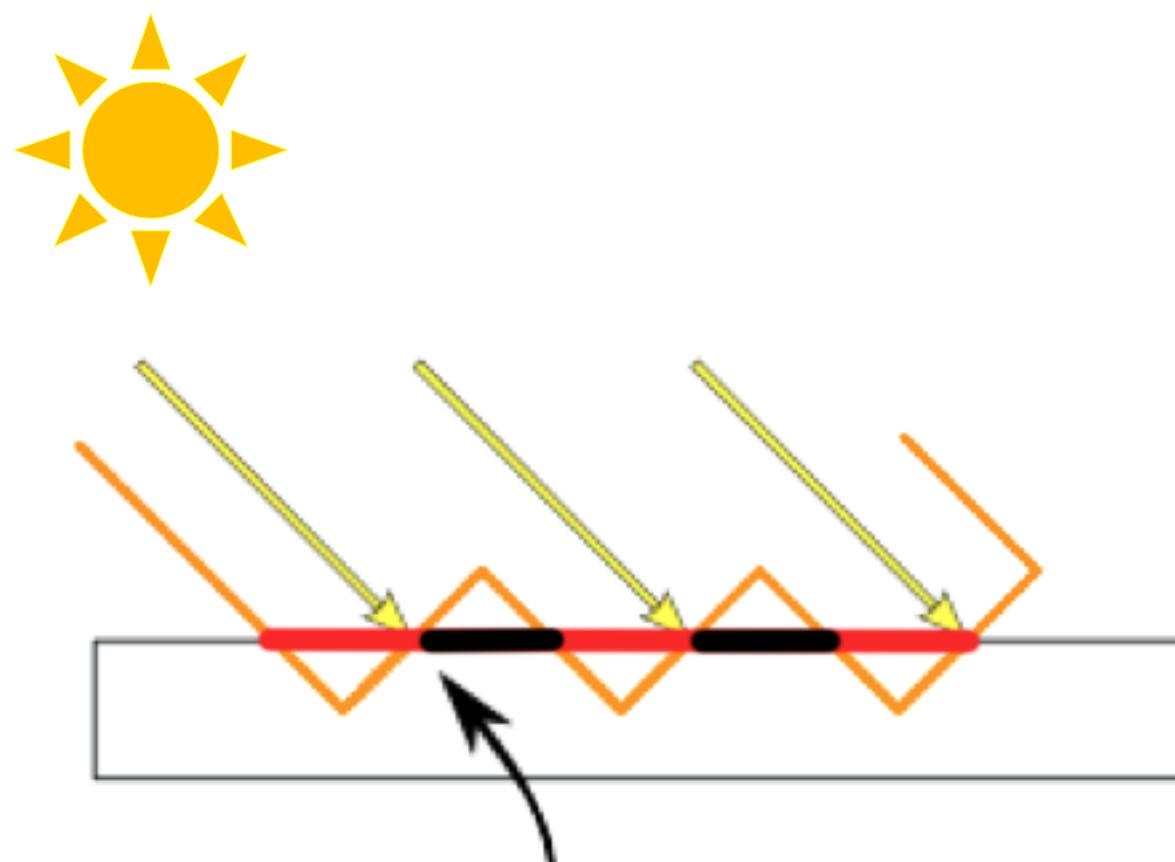
1. Взять 4 (или более) соседних текселей
2. Выполнить сравнения, результатами которых будут:
 - 0 (false, shadowPos дальше от источника, чем shadowMap)
 - или
 - 1 (true, shadowPos на расстоянии shadowMap)
3. Усреднить результаты

PCF – не обычная билинейная фильтрация, так как промежуточная выборка происходит по пороговому значению

На современном оборудовании РСФ имеет аппаратную поддержку:

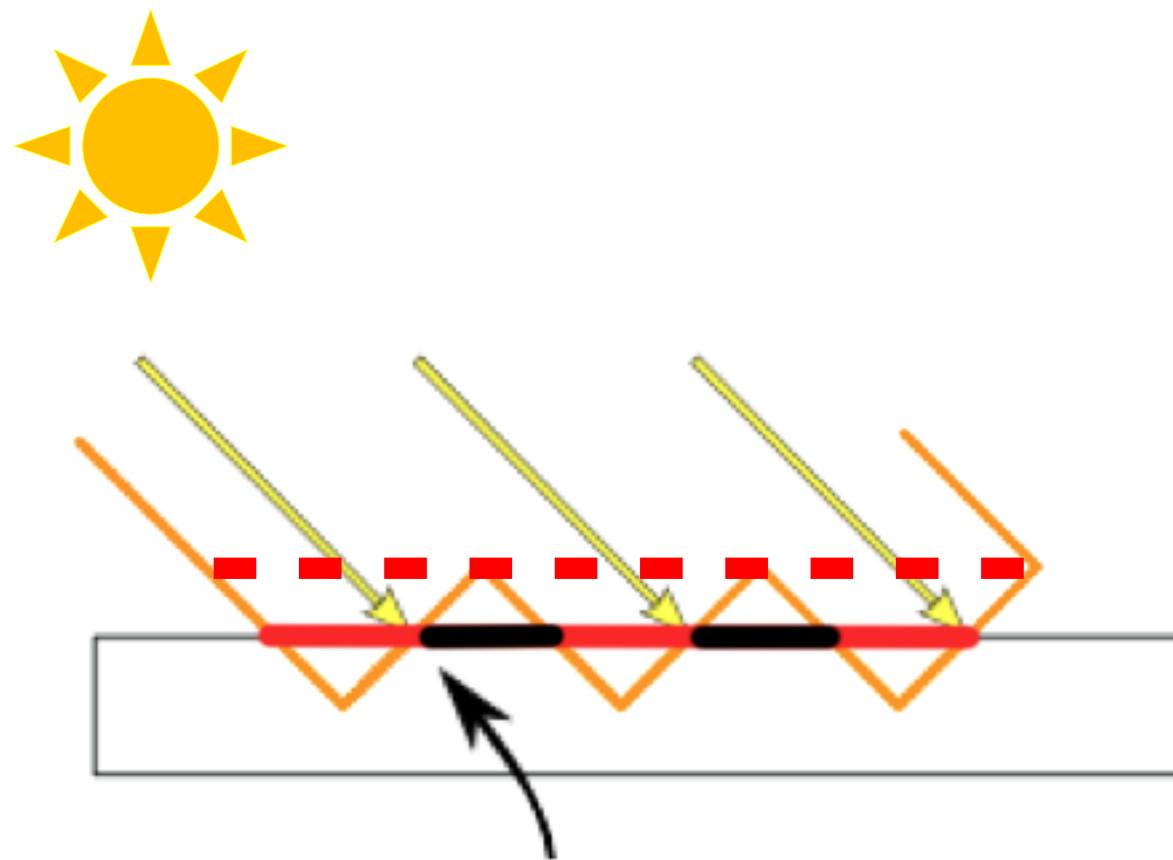
```
uniform sampler2DShadow u_shadowMap;  
in vec3 v_shadowPos;  
.  
.  
.  
  
void main()  
{  
    .  
    .  
    .  
    float s = texture(u_shadowMap, v_shadowPos); // s = [0; 1]  
    s = max(s, 0.1); // Учёт окружающей подсветки  
    fragColor = vec4(color * s, 1.0);  
}
```





Тексели текстуры тени

Дискретность текстуры тени формирует ступенчатость функции расстояния



Тексели текстуры тени

**Классический способ борьбы с теневой сыпью –
смещение глубины shadowPos, чтобы обеспечить безопасную
 ϵ -окрестность сравнения**

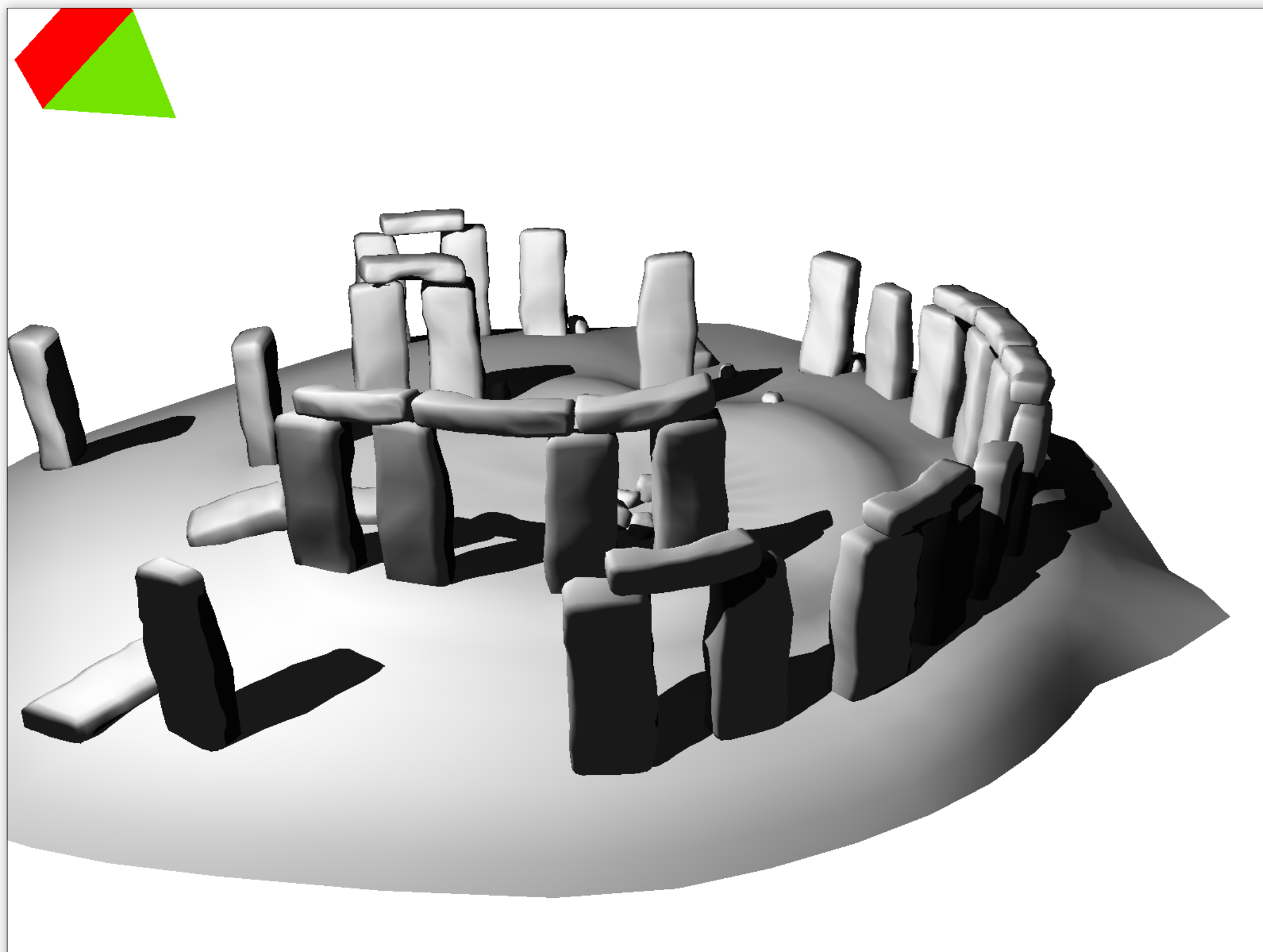
Для достижения наиболее качественного результата при смещении принимается во внимание угол падения света

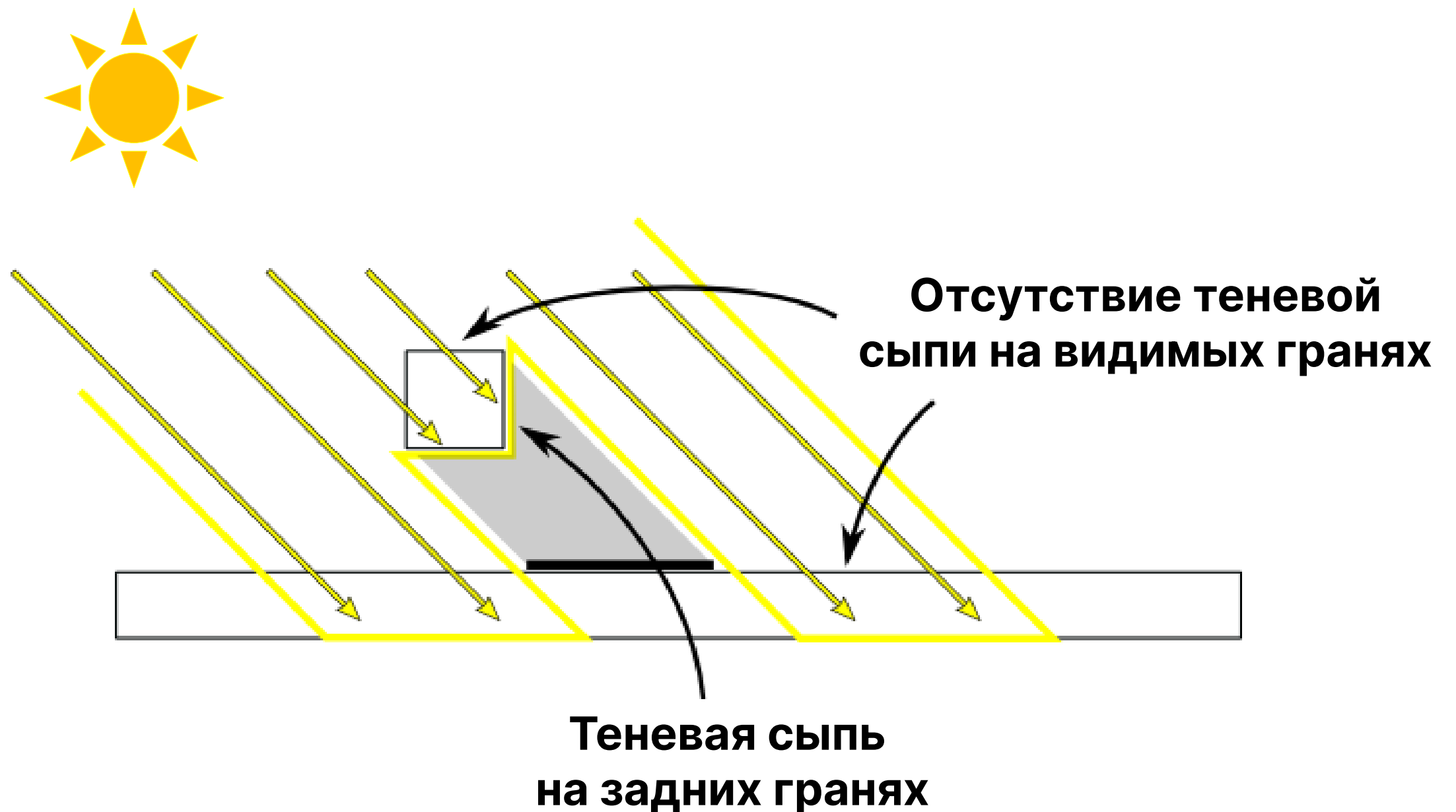
В общем случае смещение выражается так:

$$\text{bias} = a * f(\text{dot}(\mathbf{v_ray}, \mathbf{v_normal}))$$

Например:

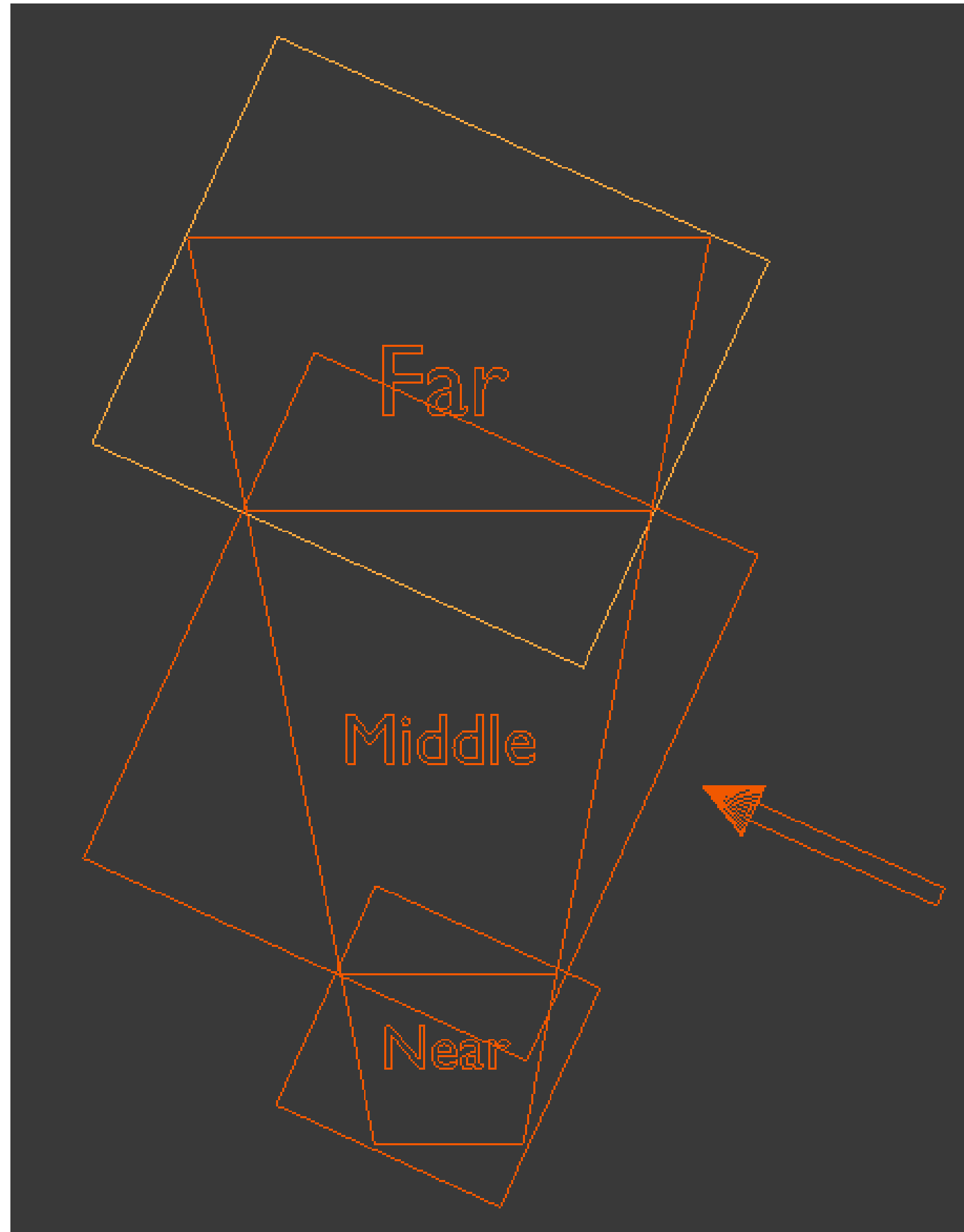
```
float c = dot(v_ray, v_normal);  
float b = max(c, 0.0);  
float bias = 0.01 * sqrt(1.0 - b * b) / b;  
bias = clamp(bias, 0.0, 0.01);  
float lightness = texture(u_shadowMap, v_shadowPos - vec3(0.0, 0.0, bias));
```





Для этого все объекты должны быть замкнутыми

- 1. Создается относительно позиции источника света, а не камеры**
- 2. Использует матрицу ортогональной проекции, плохо соотносящуюся с перспективной проекцией камеры**
- 3. Чем больше покрываемая теневой картой область, тем хуже качество теней**



Для увеличения качества подход на основе карт теней имеет множество модификаций:

- **Усложнение выборки с целью сглаживания границы:**
 - ♦ **Взятие большего числа соседей в RCF**
 - ♦ **Случайные смещения при выборе соседей**
- **Адаптивная подстройка области видимости источника света, чтобы не упереться в недостаточное разрешение текстуры тени**
- **Хранение расстояний в нелинейном виде, чтобы избежать проблем с точностью**